

PART A

Experiment No.06

A.1 Aim: To Implement 8 queen puzzle

A.2 Prerequisite: Discrete structure

A.3 Outcome:

After successful completion of this experiment students will be able to

- Apply ideas of, problem solving by searching in AI
- **Tools Required:** Python

A.4 Theory:

The [eight queens puzzle](#), or the eight queens problem, asks how to place eight queens on a chessboard without attacking each other. If you never played chess before, a queen can move in any direction (horizontally, vertically and diagonally) any number of places. In the next figure, you can see two queens with their attack patterns:

At the end of the article we present a Python 3 solution to the eight queens puzzle.

We can generate a solution to the problem by scanning each row of the board and placing one queen per column, while checking at every step, that no two queens are in the line of attack of the other. A brute force approach to the problem will be to generate all possible combinations of the eight queens on the chessboard and reject the invalid states. How many combinations of 8 queens on a 64 cells chessboard are possible ?

The [combinations formula](#) is

$$C(n,k)=n!/(k!(n-k)!) \quad C(n,k)=n!/(k!(n-k)!)$$

which, for our particular case is:

$$C(64,8) = \frac{64!}{8!(64-8)!} = 4,426,165,368$$

Clearly, the brute force approach is not practical!

We can further reduce the number of potential solutions if we observe that a valid solution can have only one queen per row, which means that we can represent the board as an array of eight elements, where each entry represents the column position of the queen from a particular row. Take as an example the next solution of the problem:

The queens positions on the above board, can be represented as the occupied positions of a two dimensional 8x8 array: [0, 6], [1, 2], [2, 7], [3, 1], [4, 4], [5, 0], [6, 5], [7, 3]. Or, as described above, we can use a one dimensional 8 elements array: [6, 2, 7, 1, 4, 0, 5, 3].

If we look closely at the example solution [6, 2, 7, 1, 4, 0, 5, 3], we note that a potential solution to the eight queens puzzle can be constructed by generating all possible permutations of an array of eight numbers, [0, 1, 2, 3, 4, 5, 6, 7], and rejecting the invalid states (the ones in which any two queens can attack each other). The number of all permutations of n unique objects is $n!$, which for our particular case is:

$$n! = 8! = 40,320$$

which is more reasonable than the previous 4,426,165,368 situations to analyze for the brute force approach.

A slightly more efficient solution to the puzzle uses a recursive approach: assume that we've already generated all possible ways to place k queens on the first k rows. In order to generate the valid positions for the $k+1$ queen we place a queen on all columns of row $k+1$ and we reject the invalid states. We do the above steps until all eight queens are placed on the board. This approach will generate all 92 distinct solutions for the eight queens puzzle.

Algorithm

isValid(board, row, col)

Input: *The chess board, row and the column of the board.*

Output – True when placing a queen in row and place position is a valid or not.

Begin

```
    if there is a queen at the left of current col, then
        return false
    if there is a queen at the left upper diagonal, then
        return false
    if there is a queen at the left lower diagonal, then
        return false;
    return true //otherwise it is valid place
```

End

solveNQueen(board, col)

Input – The chess board, the col where the queen is trying to be placed.

Output – The position matrix where queens are placed.

Begin

```
    if all columns are filled, then
        return true
    for each row of the board, do
        if isValid(board, i, col), then
            set queen at place (i, col) in the board
            if solveNQueen(board, col+1) = true, then
                return true
            otherwise remove queen from place (i, col) from
board.
        done
    return false
```

End

PART B

(PART B : TO BE COMPLETED BY STUDENTS)

Roll. No.: C-54	Name: Quazi Md Moinuddin
Class: TE Comps C	Batch: C3
Date of Experiment:25.02.25	Date of Submission:19.03.25
Grade:	

B.1 Software Code written by student:

```
# Function to print the chessboard
def print_board(board):
    for row in board:
        print(" ".join("Q" if cell else "." for cell in row))
    print()

# Check if a queen can be safely placed at board[row][col]
def is_safe(board, row, col, n):
    # Check the column
    for i in range(row):
        if board[i][col] == 1:
            return False

    # Check the upper-left diagonal
    for i, j in zip(range(row - 1, -1, -1), range(col - 1, -1, -1)):
        if board[i][j] == 1:
            return False

    # Check the upper-right diagonal
    for i, j in zip(range(row - 1, -1, -1), range(col + 1, n)):
        if board[i][j] == 1:
            return False

    return True
```

```
# Function to solve the N-Queens problem
def solve_n_queens(board, row, n):
    if row >= n:
        print_board(board)
        return True

    res = False
    for col in range(n):
        if is_safe(board, row, col, n):
            board[row][col] = 1 # Place queen
            res = solve_n_queens(board, row + 1, n) or res
            board[row][col] = 0 # Backtrack

    return res

# Driver function
def solve_8_queens():
    n = 8 # Size of the chessboard (8x8)
    board = [[0 for _ in range(n)] for _ in range(n)] # Create an empty chessboard

    if not solve_n_queens(board, 0, n):
        print("Solution does not exist")
    else:
        print("Solutions exist")

# Run the function to solve 8-Queens problem
solve_8_queens()
```

B.2 Input and Output:

```

. . . . . Q
. . . Q . . .
Q . . . . .
. . Q . . . .
. . . . . Q .
. Q . . . . .
. . . . . Q .
. . . . Q . .

```

Solutions exist

B.3 Observations and learning:

1. **Backtracking Approach:** The solution uses backtracking, where queens are placed row by row. If placing a queen leads to a conflict, it backtracks and tries the next column.
2. **Safety Check:** The `is_safe()` function ensures no queen can attack another by checking columns and diagonals.
3. **Efficiency:** The algorithm has a time complexity of $O(N!)$ (due to recursion and backtracking) and a space complexity of $O(N^2)$ (for the board and recursion stack).
4. **Single Solution:** The program prints only the first valid solution, but the problem has **92 distinct solutions** for an 8x8 board.
5. **Constraint Satisfaction:** The problem is a constraint satisfaction problem (CSP) where the goal is to place queens without violating the constraints (no two queens in the same row, column, or diagonal).

B.4 Conclusion:

The 8-Queens problem is solved using backtracking, efficiently finding one valid solution by placing queens row by row while ensuring no two queens can attack each other. The solution has a time complexity of $O(N!)$ and space complexity of $O(N^2)$. While the problem has 92 solutions, the code finds and prints the first one. This showcases how backtracking can be applied to constraint satisfaction problems to explore solution spaces effectively.

B.5 Question of Curiosity

1) How to formulate a problem?

How to Formulate a Problem:

1. **Understand the Context:** Identify the domain and gather relevant background information.
2. **Define the Problem:** Clearly state what the problem is and what you aim to achieve.
3. **Identify Constraints and Requirements:** List limitations (e.g., time, budget) and conditions for a solution.
4. **Break Down the Problem:** Decompose the issue into smaller parts and define inputs/outputs.
5. **Assumptions and Variables:** Identify assumptions and key variables influencing the solution.
6. **Propose Solutions:** List possible solutions and evaluate them based on feasibility.
7. **Test and Refine:** Pilot test the chosen solution and refine it as needed.

2) State space representation of the problem?

State Space Representation of a Problem:

State space representation is a way to model a problem in terms of its possible states and the transitions between them. Here's how to define it:

1. **State:** A unique configuration or condition of the problem at a particular point in time (e.g., board arrangement in the 8-Queens problem).
2. **Initial State:** The starting point or configuration of the problem (e.g., an empty chessboard in the 8-Queens problem).
3. **Actions:** Operations or moves that change the state (e.g., placing a queen in a valid position on the board).
4. **Transition Function:** Defines how actions lead to new states (e.g., placing a queen leads to a new arrangement of queens).
5. **Goal State:** The final configuration that satisfies the problem's objectives (e.g., all queens placed without conflicts).
6. **State Space:** The complete set of all possible states, including the initial state and goal state, that can be reached through valid actions.

Example (8-Queens Problem):

- **State:** A specific arrangement of queens on the chessboard.
- **Initial State:** No queens placed on the board.
- **Actions:** Placing a queen in each row.
- **Goal State:** All 8 queens placed on the board such that no two queens attack each other.